

ASSIGNMENT-1

Section - A

1. Convert the decimal number 53 to binary, octal, hexadecimal representations.

→ $(53)_{10} \rightarrow (?)_2$

2	53	1
2	26	0
2	13	1
2	6	0
2	3	1
	1	

↑

→ $(110101)_2$

$(53)_{10} \rightarrow (?)_8$

8	53	5
8	6	6
	10	

↑

→ $(65)_8$

$(53)_{10} \rightarrow (?)_{16}$

$53 / 16 \rightarrow Q = 3 \quad R = 5$

$3 / 16 \rightarrow Q = 0 \quad R = 3$

Ans = $(35)_{16}$

2. Define the term 'bus' in the context of computer architecture and explain its types.

A bus is a set of electrical wires that connects the various hardware components of a computer system. It has 3 comp components:

- Data bus: used to transmit data.



- Control bus: transfers the control and timing signals from one component to another.
- Address bus: used to carry address from CPU to memory/I/O devices.

3. What is the role of ALU in CPU?

→ ALU (Arithmetic Logic Unit) is the main component of CPU which performs arithmetic and logic operations. It has the ability to perform all processes related to $+$, $-$, shifting operations, boolean comparisons, etc.

4. Find R' complement of:

a) $(11011)_2$

→ $x^n - N$ $x=2, n=5, N=11011$

$2^5 - 11011$

$(32)_{10} - (11011)_2 \rightarrow 100000 - 11011$

100000

$- 011011$

000101

Ans = 101

b) $(BAC)_{16}$

→ $x^n - N$ $x=16, n=3, N=BAC$

$16^3 - BAC$

$4096 - 2988$

$= 1108$

→ $(454)_{16}$

$\therefore (BAC)_{16} = (2988)_{10}$

c) $(703)_8$

→ $x^n - N$ $x=8, n=3, N=703$

$8^3 - 703$

$(512)_{10} - 703$

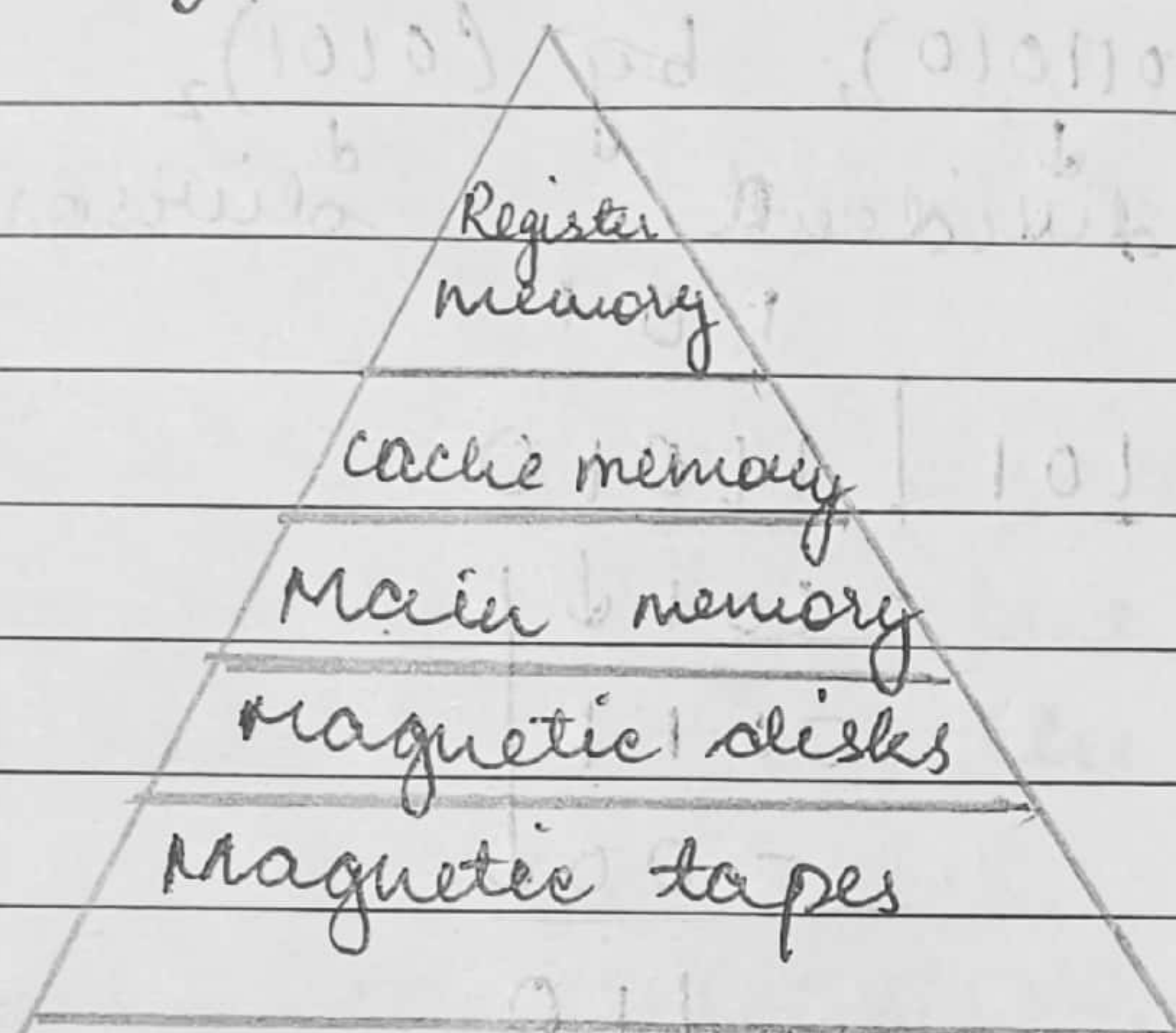
$(1000)_8 - (703)_8 \rightarrow 207$

$$\begin{array}{r}
 777 \\
 - 703 \\
 \hline
 074 \rightarrow 7\text{'s complement } (2-1) \\
 + 1 \\
 \hline
 075 \rightarrow 8\text{'s complement}
 \end{array}$$

Aus = 75

5. What is the purpose of memory hierarchy in computer architecture? Briefly explain.

→ Memory hierarchy in computer architecture shows the organization of memory so that it can minimize the access time.



↑ Increasing order of access time.

Section - B

1. Explain how to perform division using binary numbers. Provide an example.

→ Rules for binary division.

Dividend	Divisor	Result
0	1	0
1	1	1

Division by 0 is meaningless.

Steps:

- compare divisor with dividend. If divisor is large, place 0 as quotient then bring 2nd bit of dividend down. If the divisor is smaller, multiply with 1 and subtract the numbers.
- Then bring down the next no. bit from dividend portion and perform 1st step again.
- Repeat same process until remainder becomes 0 or whole dividend is divided.

eg- Divide $(011010)_2$ by $(0101)_2$.

$\downarrow$ dividend $\downarrow$ divisor
 101

$ \begin{array}{r} 101 \overline{) 11010} \\ \underline{- 101} \\ 0011 \\ \underline{- 000} \\ 110 \\ \underline{- 101} \\ 001 \end{array} $	Subtraction Rules $0-1 = 1$ (borrow 1) $1-1 = 0$ $1-0 = 0$ $0-0 = 0$ Multiplication Rules $1 \times 1 = 1$ $0 \times 0 = 0$ $0 \times 1 = 0$ $1 \times 0 = 0$
--	--

Quotient $\rightarrow (101)_2$

Remainder $\rightarrow (001)_2$

2. Compare and contrast Harvard and Von Neumann architecture.

Von Neumann	Harvard
• It's an ancient computer architecture based on stored program computer.	• It is modern computer architecture based on Harvard Mark I relay.

concept.

based model.

- Same physical memory address is used for instructions and data.

- Separate physical memory address is used for instructions and data.

- There is common bus for data and instruction transfer.

- Separate buses are used for transferring data and instruction.

- Two clock cycles are used to execute single instruction.

- Instructions are executed in a single cycle.

- It is cheaper in cost.

- It is expensive.

- CPU can't access instructions and read/write at the same time.

- CPU can access instructions and read/write at the same time.

- Used in personal and small computers.

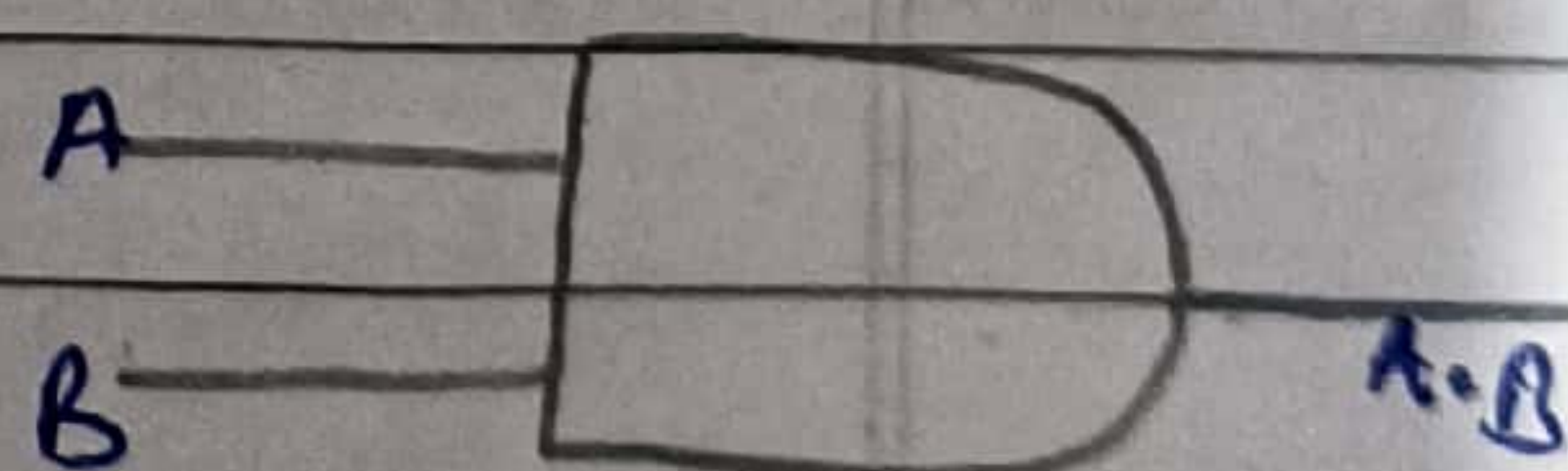
- Used in microcontroller and signal processing.

3.a) Define the basic logic gates (AND, OR, NOT, XOR) and explain their truth tables.

→ • AND gate - performs logical multiplication of inputs applied to it.

A	B	A AND B
0	0	0
0	1	0
1	0	0
1	1	1

Symbol:



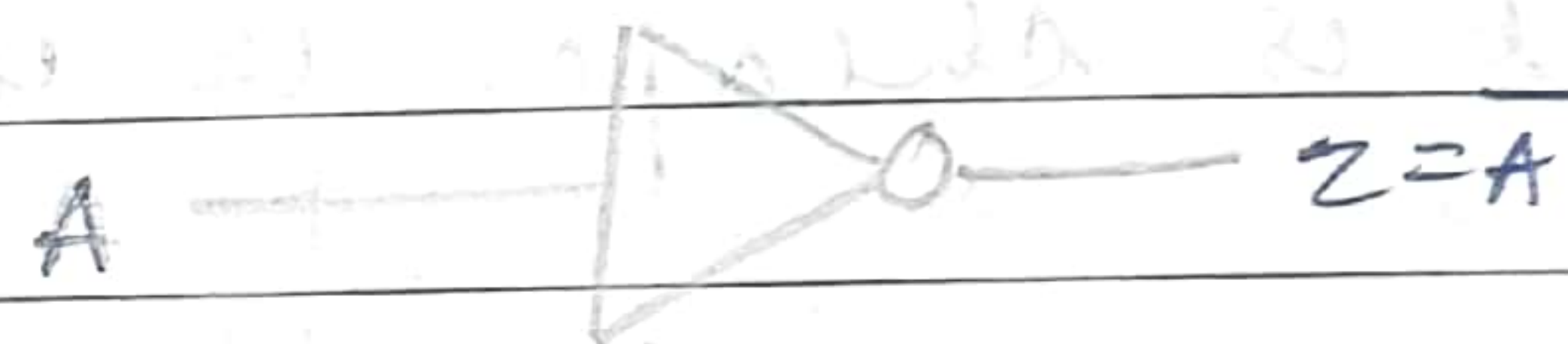
- OR gate: produces low or logic 0 output only when all its inputs are low or logic 0.

A	B	$Z = A + B$
0	0	0
0	1	1
1	0	1
1	1	1



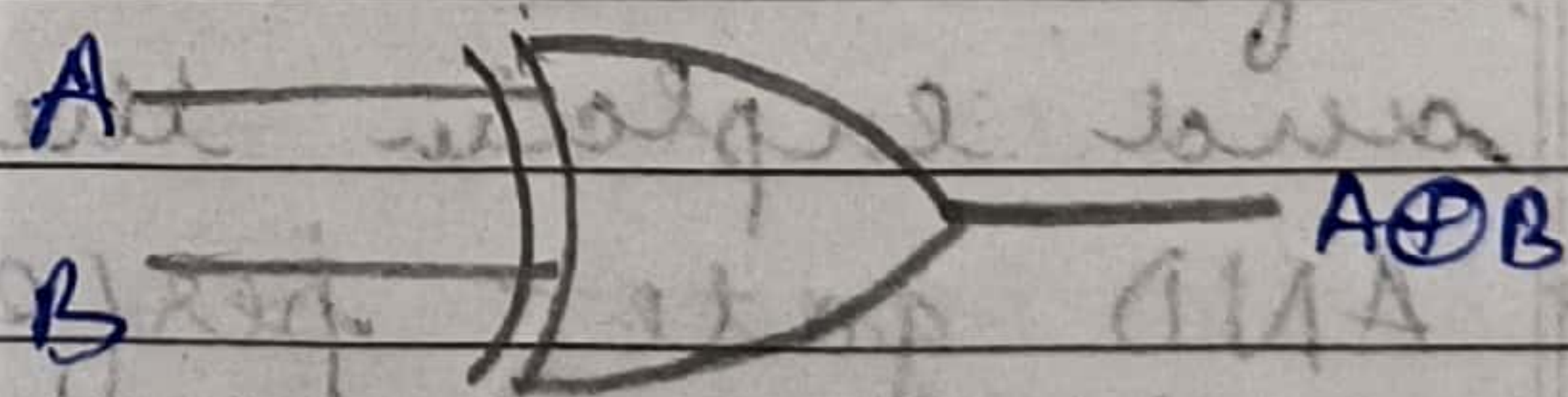
- NOT gate: this logic gate performs the complement of an input signal applied to it. It takes only one input and one output.

A	$\text{NOT } A$
0	1
1	0



- XOR gate: also called Exclusive-OR gate or Ex-OR gate; used to perform modulo sum. It takes only 2 inputs and gives an output.

A	B	$A \oplus B$	$A \oplus B = A\bar{B} + \bar{A}B$
0	0	0	
0	1	1	
1	0	1	
1	1	0	



- b) Construct a truth table for expression: $(A \text{ AND } B) \text{ OR } (\text{NOT } A)$

A	B	$A \cdot B = C$	$D = \bar{A}$	$C + D$
0	0	0	1	1
0	1	0	1	1
1	0	0	0	0
1	1	1	0	1

Section - C

1. Discuss the significance of signed-magnitude representation in computer arithmetic. Also discuss the advantages of using float point representation over fixed point representation in numerical computations.

→ Signed magnitude representation helps us to represent positive and negative numbers in binary representation. In this method of representing integers, MSD (Most Significant Digit) takes on extra meaning.

If MSD is 0 → +ve.

If MSD is 1 → -ve.

g→

+6	-6
0110	1110
MSD representation of 6	MSD representation of 6

Binary → (6)₁₀ = (110)₂

This representation tells us whether the number is +ve or -ve.

ADVANTAGES

- **Dynamic range:** float point representation allows for a much wider range of numbers to be represented as compared to fixed point representation.
- **Flexibility:** it allows for scaling of numbers, i.e., decimal can be moved to accommodate a wide range of magnitudes whereas fixed point requires fixed scaling.

- Precision: more precise. This is actually needed in scientific and engineering calculations whereas accuracy is crucial.
- Ease of use: operations $(+, -, *, /)$ are often implemented in hardware and software making them easier to use and more efficient.
- Standardization: follows formats like IEEE 754 which simplifies numerical computations and promotes portability of code.

2. What is number system? Define all types and convert the following.

A a) $(1101101)_2 = (?)_{16}$

b) $(256.51)_8 = (?)_{10}$

c) $(AB10)_{16} = (?)_2$

→ A number system is an elementary system and a unique way of representing numbers in arithmetic and algebraic structure.

TYPES

- Decimal number system $(?)_{10}$:
 → It has a base value of 10 with digits 0-9.
- Binary number system $(?)_2$:
 → It has a base value of 2 with digits 0, 1.
 These numbers are called binary numbers.
 eg- $(14)_{10} = (1110)_2$

- Octal number system $()_8$
- It has a base value of 8 with digits 0-7.

eg - $(87)_{10} = (121)_8$

- Hexadecimal number system $()_{16}$:

- It has a base value of 16 with digits 0-9 and alphabets A-F.

10 - A

11 - B

12 - C

13 - D

14 - E

15 - F

a) $(1101101)_2 = (?)_{16}$

By using grouping:

$0110 \rightarrow 2^2 + 2^1 = 6$

$1101 \rightarrow 2^3 + 2^2 + 2^0 = 8 + 4 + 1 = 13 = D$

$(1101101)_2 = (6D)_{16}$

b) $(256.51)_8 = (?)_{10}$

→ $(2 \times 8^2) + (5 \times 8^1) + (6 \times 8^0) + (5 \times 8^{-1}) + (1 \times 8^{-2})$

→ $128 + 40 + 6 + 0.625 + 0.015625$

→ 174.640625

c) $(AE10)_{16} = (?)_2$

A = 1010

E = 1110

1 = 0001

0 = 0000

→ $(AE10)_{16} = (1010111000010000)_2$

3. Perform addition / subtraction on following (binary)

a) $(100110)_2 + (1101)_2$

$$\begin{array}{r} 100110 \\ + 001101 \\ \hline 110011 \end{array}$$

→ Ans

b) $(11100)_2 - (1111)_2$

$$\begin{array}{r} 11100 \\ - 01101 \\ \hline 01101 \end{array}$$

c) $(-18) + (-67)$
 $-18 - 67 \Rightarrow -145$
 $(-145)_{10} \rightarrow (?)_2$

$(1110010001)_2$

$1 = 2^0 = 1 \times 1 = 1$
 $1 = 2^1 = 2 \times 1 = 2$
 $0 = 2^2 = 0 \times 1 = 0$
 $0 = 2^3 = 0 \times 1 = 0$
 $1 = 2^4 = 1 \times 16 = 16$
 $0 = 2^5 = 0 \times 32 = 0$
 $0 = 2^6 = 0 \times 64 = 0$
 $1 = 2^7 = 1 \times 128 = 128$
 $1 = 2^8 = 1 \times 256 = 256$
 $1 = 2^9 = 1 \times 512 = 512$
 $1 = 2^{10} = 1 \times 1024 = 1024$

$(1110010001)_2 = 1024 + 512 + 256 + 128 + 16 + 1 = 1937$

$(10110101)_2 = 16 + 8 + 4 + 2 + 1 = 27$

$(101101010101)_2 = 16384 + 8192 + 4096 + 2048 + 1024 + 512 + 256 + 128 + 64 + 32 + 16 + 8 + 4 + 2 + 1 = 32768$